LSTM & Sequence Modeling

Structured Notes

From TF-IDF Limitations to Vanishing Gradients
and Gated Memory in Long Short-Term Memory Networks

Contents

1 From TF-IDF to Sequence Models **2**
- 1.1 Recap: TF-IDF with Classical ML . . . 2
- 1.2 What TF-IDF Captures Well . . . 2
- 1.3 What TF-IDF Misses . . . 2
- 1.4 Static vs. Dynamic Representation . . . 2
- 1.5 Why This Matters for Deep Learning NLP . . . 3
- 1.6 Visualizing TF-IDF Limitations . . . 3

2 Vanilla RNN and Its Limitations **4**
- 2.1 Plain (Vanilla) RNN . . . 4
- 2.2 Known Limitations of Vanilla RNN . . . 4
- 2.3 Why Vanilla RNN Fails on Long Dependencies . . . 4
  - 2.3.1 Vanishing Gradients from tanh . . . 4
  - 2.3.2 Exploding Gradients from the Recurrent Jacobian . . . 4
- 2.4 Role of tanh Saturation . . . 4
- 2.5 Which Is More Common: Vanishing or Exploding? . . . 5
- 2.6 Interpretation: Fragile Memory . . . 5
- 2.7 Overfitting in RNN . . . 5

3 Visualizing tanh, Its Derivative, and Gradient Dynamics **5**
- 3.1 tanh vs. Its Derivative . . . 5
- 3.2 Vanishing vs. Exploding Gradient Over Time . . . 5
- 3.3 Toy Recurrence: State Compression . . . 6

4 Long-Term Dependency: The Formal Picture **6**
- 4.1 Why Gradients Must Pass Through Many Time Steps . . . 6
- 4.2 Why Repeated Multiplication Vanishes the Gradient . . . 6

5 Long Short-Term Memory (LSTM) **6**
- 5.1 Motivation: Protected Memory Path . . . 6
- 5.2 Two States in LSTM . . . 6
- 5.3 Gating: The Restaurant Parking Analogy . . . 7

6 LSTM Gates and Update Equations **7**
- 6.1 Step 1: Forget Gate . . . 7
- 6.2 Step 2: Input Gate . . . 7
- 6.3 Step 3: Candidate Memory . . . 7
- 6.4 Step 4: Cell State Update . . . 8
- 6.5 Why the Additive Update Matters . . . 8
- 6.6 Step 5: Output Gate . . . 8

7 Summary of LSTM Components **9**

8 Limitations of LSTM and What Comes Next **9**

9 Concept Check **9**

10 Summary: Key Points **9**

1 From TF-IDF to Sequence Models

1.1 Recap: TF-IDF with Classical ML

A sentiment analysis pipeline using TF-IDF plus a classical machine learning model (e.g., logistic regression, SVM, or Naive Bayes) converts each document or sentence into a fixed-length feature vector. Each component of this vector measures the importance of a word in a document relative to the corpus, or relative to the sentence. This approach is powerful for many classification tasks.

The Deeper Question

Does a document's meaning depend only on which words appear, or also on the order in which they appear? Order matters — meaning is carried not just by words but by how they are arranged.

1.2 What TF-IDF Captures Well

TF-IDF is useful when:

- Word importance matters, not the order.
- Document-level classification is the main goal (classifying the complete sentence, order is less critical).
- Interpretable sparse features are desired.

For example, in sentiment classification, words such as *excellent*, *amazing*, *worst*, and *poor* can already provide strong signal. TF-IDF is not wrong; it is simply limited to words only. It captures the main words, assigns importance based on them, and processes sentiment classification. Accuracy obtained with this approach is around 75–80%, and not much more than that, because we are classifying without considering order — focusing only on the main words.

1.3 What TF-IDF Misses

TF-IDF treats text mostly as a **bag of weighted words**. As a result:

- Two sentences such as *"not good"* and *"good"* become very similar representations because the vocabulary overlap is high and the weighting does not encode the modifier.
- Consider *"dog bites man"* versus *"man bites dog"*. Under TF-IDF the meaning is lost — the vectors are identical because the same words appear.
- TF-IDF does not naturally model word order, local context, evolving meaning across positions, or dependence of later words on earlier words.

Basic Vectorization vs. Embeddings

Basic vectorization, which TF-IDF uses, is similar to one-hot encoding. One-hot encoding also creates sparse vectors; the representation is different (0s and 1s vs. weighted entries), but both produce sparse vectors. This is very different from embeddings, which create **dense** vectors.

1.4 Static vs. Dynamic Representation

In TF-IDF, the whole sentence or document is compressed into one fixed vector. From the complete document, a vocabulary is created, and every sentence is compressed into the length of that vocabulary. That is **static representation**.

In sequence models, we process tokens one step at a time and build an internal evolving representation (recall h_t, memory). Meaning is no longer treated as one frozen vector. Instead, meaning is updated through time as the sequence unfolds.

Sequence intuition. Consider: *"I thought the movie would be great, but it was disappointing."* If we only count important words, both *great* and *disappointing* appear. The actual sentiment depends

> on how meaning develops across the sequence: first expectation is positive, later reversal happens with *but*, and the final interpretation becomes negative.
>
> In sequence modeling, meaning is not just present in words; it is present in the **trajectory of words**.

1.5 Why This Matters for Deep Learning NLP

Many NLP tasks require order-sensitive understanding:

- Next-word prediction
- Machine translation
- Text generation
- Speech processing
- Sequence labeling
- Question answering
- Summarization

These tasks require a model to remember what came before and use it while processing what comes next. For these tasks, TF-IDF fails miserably because it does not capture context, does not model what comes back or what comes later. This motivates recurrent sequence models.

1.6 Visualizing TF-IDF Limitations

A simple experiment vectorizes four sentences: *good movie*, *not good movie*, *dog bites man*, *man bites dog*. The resulting TF-IDF vectors reveal that representations are **strong on word presence but weak on order**. For *good movie*, only *good* and *movie* carry weight. For *not good movie*, *not* carries the highest weight (as the rare first word) followed by *good* and *movie*. For *dog bites man* and *man bites dog*, the vectors carry identical weights on *dog*, *bites*, and *man* — the two sentences are indistinguishable.

Illustrating TF-IDF's Order Blindness

```python
from sklearn.feature_extraction.text import TfidfVectorizer
import matplotlib.pyplot as plt
import numpy as np

sentences = [
    "good movie",
    "not good movie",
    "dog bites man",
    "man bites dog"
]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(sentences).toarray()
vocab = vectorizer.get_feature_names_out()

# Plot weights for each sentence over the shared vocabulary.
fig, axes = plt.subplots(4, 1, figsize=(7, 8))
for ax, row, sent in zip(axes, X, sentences):
    ax.bar(vocab, row, color="steelblue")
    ax.set_title(sent)
plt.tight_layout()
plt.show()
```

Listing 1: TF-IDF vectorization across four sentences

TF-IDF works reasonably well for applications like **sentiment analysis** and **spam detection** (because spam detection needs only keywords), but it is fundamentally limited for **next-word prediction** (e.g., chatbots) and any task requiring order sensitivity.

2 Vanilla RNN and Its Limitations

2.1 Plain (Vanilla) RNN

The basic RNN — the one with a hidden state that updates at every time step — is called a **plain RNN** or **vanilla RNN**.

Vanilla RNN update rule:

$$h_t = \tanh\left(W_{xh}x_t + W_{hh}h_{t-1} + b\right)$$

The same function (layer) recurs again and again; h_t is updated at every time step. There is no sequence of different models — the same layer appears at different time stamps.

2.2 Known Limitations of Vanilla RNN

- **Fixed-size memory.** The hidden state has fixed dimensionality, and all past information must be compressed into it.
- **Same function recurring.** The same transformation is applied at every step, so there is no explicit mechanism to decide what to keep or forget.
- **Long-term dependency failure.** When sequences grow long, context from early tokens is lost by the time processing reaches the last token, because memory is time-updated and continually overwritten.

2.3 Why Vanilla RNN Fails on Long Dependencies

2.3.1 Vanishing Gradients from tanh

The derivative of $\tanh(z)$ is

$$\frac{d}{dz}\tanh(z) = 1 - \tanh^2(z),$$

which always lies in $[0, 1]$. When $\tanh(z)$ is saturated near -1 or $+1$, the derivative becomes very small. If we repeatedly multiply such small factors during back-propagation through time, the overall gradient repeatedly shrinks.

2.3.2 Exploding Gradients from the Recurrent Jacobian

The repeated dot product does not always shrink. If the effective norm of the recurrent Jacobian is greater than 1, repeated multiplication can make gradients grow very large. Gradient explosion depends not only on tanh but also on the weight matrix.

Two Opposing Risks

Vanilla RNN training suffers from two opposite failure modes:

- **Vanishing gradient** — gradient becomes too small to learn long-range dependencies.
- **Exploding gradient** — gradient becomes too large and destabilizes training.

2.4 Role of tanh Saturation

The problem is not only the weight matrix; the nonlinearity also matters. For large positive or large negative z, $\tanh(z) \approx \pm 1$, and therefore $1 - \tanh^2(z) \approx 0$. This means the neuron output changes very little even if the input changes. In saturated regions, forward information becomes compressed and backward gradients become weak. This makes it difficult for the model to preserve long-term effects.

- Gradients **explode** due to the Jacobian / weight matrix norm.
- Gradients **vanish** due to the tanh derivative.

2.5 Which Is More Common: Vanishing or Exploding?

Vanishing gradient is more prone to occur in vanilla RNN. The reasoning:

- Suppose the weight matrix norm is greater than 1: multiplication pushes gradients toward explosion, but as the sequence lengthens, tanh saturates and its derivative moves toward 0, balancing out the growth.
- If the weight matrix norm is less than 1 *and* tanh saturates, both factors push toward 0 — the gradient vanishes.

Therefore vanilla RNN **can suffer from both**, but is **more prone to vanishing gradient**.

2.6 Interpretation: Fragile Memory

Memory in vanilla RNN is fragile — it can break down very easily. Vanilla RNN *does* have memory, but it is unstable memory:

- Remembered information must survive repeated nonlinear transformations.
- There is no explicit mechanism saying "keep this information," "forget that information," or "protect this signal from being overwritten."
- Everything is mixed into the same hidden state update — no filtering.

Information must pass through many recurrent steps, and so must gradients. Repeated multiplication causes shrinking or amplification; tanh saturation makes long-range learning even harder. We therefore need a recurrence mechanism with better gradient flow and more controlled memory.

2.7 Overfitting in RNN

If an RNN overfits, it fails to generalize: for any input sentence it tends to predict a similar kind of word after every input. For example, after *I*, it might always predict *am I love I am . . . I am I am I am.* That is what overfitting in RNN looks like.

3 Visualizing tanh, Its Derivative, and Gradient Dynamics

3.1 tanh vs. Its Derivative

Plotting tanh and its derivative

```python
import numpy as np
import matplotlib.pyplot as plt

z = np.linspace(-5, 5, 400)
tanh_z = np.tanh(z)
dtanh_z = 1 - tanh_z**2

plt.plot(z, tanh_z, label="tanh(z)")
plt.plot(z, dtanh_z, label="1 - tanh^2(z)")
plt.axhline(0, color="gray", linewidth=0.5)
plt.legend()
plt.show()
```

Listing 2: ; derivative range [0, 1]]tanh range [-1, 1]; derivative range [0, 1]

- $\tanh(z)$ is zero-centered and ranges in $[-1, 1]$.
- Sigmoid ranges in $[0, 1]$; its break-even point is 0.5, not 0. Sigmoid is not zero-centered.
- The derivative $1 - \tanh^2(z)$ is maximum 1 at $z = 0$ and approaches 0 as $|z|$ grows.

3.2 Vanishing vs. Exploding Gradient Over Time

Plotting gradient magnitude as a function of the number of recurrent steps:

- **Vanishing:** gradient starts near 0 and decays further toward 0 exponentially. Due to the scale, it can appear flat against an exploding curve.
- **Exploding:** gradient grows exponentially across time steps.

3.3 Toy Recurrence: State Compression

A small toy dataset run through repeated nonlinear recurrence shows how information can be compressed and washed out. The hidden value keeps decreasing with each time step, illustrating how repeated nonlinear transformation erodes state information.

4 Long-Term Dependency: The Formal Picture

Long-Term Dependency

A long-term dependency is a dependency where the prediction at a later step depends on information from much earlier steps.

4.1 Why Gradients Must Pass Through Many Time Steps

Gradients from a late loss must travel back through the full unrolled chain because of recurrence: each hidden state depends on the previous one. Back-propagation through time (BPTT) must traverse the chain backward. If a loss at time T depends on h_T, and h_T depends on h_{T-1}, then $\partial L/\partial h_k$ must pass through all intermediate time steps.

4.2 Why Repeated Multiplication Vanishes the Gradient

Multiplying many values smaller than 1 (and in $[0, 1]$) drives the product toward 0. Combined with tanh saturation — where the derivative approaches 0 — the gradient shrinks more strongly. The deeper issue is the difficulty of **optimization through time**, though the memory representation is also structurally weak.

5 Long Short-Term Memory (LSTM)

5.1 Motivation: Protected Memory Path

LSTM

LSTM (Long Short-Term Memory) was introduced as a solution to the fragile memory of vanilla RNN. The idea is to design a recurrent model where memory has a more protected path, and the model can decide **what to keep**, **what to forget**, and **what to expose**.

In basic RNN, one hidden state held everything, producing two problems:

- Memory is repeatedly overwritten.
- Gradient must pass through repeated nonlinear transformation.

LSTM key idea: do not store everything in one place. Separate internal memory from visible state, and control information flow using gates.

5.2 Two States in LSTM

At each time step, LSTM maintains two different vectors:

- **Cell state** C_t — the long-term memory channel.
- **Hidden state** h_t — the exposed working state, used for output and for interaction with the next step.

Unlike vanilla RNN, LSTM does not force all information into one hidden vector. It creates a more structured memory system.

5.3 Gating: The Restaurant Parking Analogy

At every step, the model must answer questions like:

- What old information should be kept?
- What old information should be erased?
- What new information should be written?
- What part of memory should be revealed as output?

Think of a restaurant with two types of parking: general crowd and VIP. The model needs features (pass type, number plate, etc.) to distinguish which car goes to VIP parking and which to general parking. LSTM answers these routing questions using **gates**.

Gate

A gate is typically a sigmoid-based controller. Since sigmoid outputs lie in $[0, 1]$, a gate can continuously scale information — near 0 it blocks information, near 1 it passes information through.

6 LSTM Gates and Update Equations

6.1 Step 1: Forget Gate

The forget gate decides **how much of the previous cell state should be retained** — what proportion of previous information to delete and what to keep.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Here x_t is the current input, h_{t-1} is the previous hidden state, and f_t is the forget gate vector passed through the sigmoid.

Interpretation: If a component of f_t is close to 1, keep that memory (do not touch that cell state component). If close to 0, forget that memory and rewrite it. f_t is a vector (one-dimensional, not a matrix).

6.2 Step 2: Input Gate

The input gate decides **how much of new information should be written into memory**.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

The input gate does not create content by itself. It only decides how strongly the new candidate memory should be allowed to enter. It projects importance onto the input values, much like deciding whether a customer is a VIP or not.

6.3 Step 3: Candidate Memory

The model also computes a candidate memory content using tanh.

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

This is the **new information proposal**. It is not a gate.

Interpretation: i_t decides *how much* to write, while $\tilde{C}_t$ decides *what* to write. The candidate memory

is a part of the input gate stage — after entering through the gate, what needs to be written is the candidate memory.

6.4 Step 4: Cell State Update

Now we combine the retained old memory and the gated new memory.

$$C_t = f_t \odot C_{t-1} \; + \; i_t \odot \tilde{C}_t$$

Where $\odot$ denotes element-wise multiplication (Hadamard product).

This is **the most important LSTM equation**. It says: keep the part of old memory ($f_t \odot C_{t-1}$), and add selected new memory ($i_t \odot \tilde{C}_t$). Memory is updated **additively**, not through full nonlinear replacement. Only the selected memory is written and only the selected memory is forgotten — nothing is completely overwritten and nothing is compressed through a nonlinearity alone.

Hadamard Product

Element-wise multiplication of two vectors: $[2, 3, 4] \odot [1, 0, 2] = [2, 0, 8]$. Not a matrix multiplication — just component-wise.

6.5 Why the Additive Update Matters

In vanilla RNN, the hidden state is repeatedly transformed through tanh, so information is always pushed through a nonlinearity.

In LSTM, the cell state update contains a **direct additive path**:

$$\frac{\partial C_t}{\partial C_{t-1}} \approx f_t.$$

If f_t is near 1, memory and gradient can pass through many steps much more easily. This is the core reason LSTM improves long-range learning — the gradient does not vanish due to the additive nature and the Hadamard product along the cell state channel.

6.6 Step 5: Output Gate

Not all internal memory should be exposed at every step. LSTM uses an output gate. This gate decides which part of the internal memory becomes visible in the hidden state.

$$o_t = \sigma\big(W_o \cdot [h_{t-1}, x_t] + b_o\big)$$
$$h_t = o_t \odot \tanh(C_t)$$

7 Summary of LSTM Components

Component	Equation	Role
Forget gate	$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$	How much past to keep
Input gate	$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$	How strongly to write new info
Candidate memory	$\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$	What content to write
Cell state update	$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$	Additive long-term memory
Output gate	$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$	What to expose
Hidden state	$h_t = o_t \odot \tanh(C_t)$	Visible working state

8 Limitations of LSTM and What Comes Next

LSTM helps with long-range dependency but still processes sequence information step by step. Open questions remain:

- Can recurrence parallelize efficiently?
- Can it access distant tokens more directly?
- Does a fixed state remain a bottleneck?

These questions motivate the next topic: the **Transformer**.

9 Concept Check

1. Why is TF-IDF called a static representation? — Because the whole sentence, regardless of its length, is represented in a fixed vector space based on the vocabulary.
2. Give one NLP task where TF-IDF can work reasonably well. — Sentiment analysis, spam detection.
3. Give one NLP task where TF-IDF is fundamentally limited. — Next-word prediction.
4. Why are *good movie* and *not good movie* a useful comparison? — Because similar words appear, but the sentiment/meaning changes due to order and dependency.
5. Main conceptual difference between document vector representation and sequence state representation? — Document vector is a one-shot global representation of the complete sentence; sequence state representation is dynamic and keeps evolving context over time.
6. Why do we need sequence models? — Because language meaning depends on order and context.
7. What is the derivative of $\tanh(z)$? — $1 - \tanh^2(z)$.
8. Why does tanh saturation worsen the vanishing gradient problem? — In saturation, the derivative becomes close to 0, so the gradient shrinks more strongly.
9. Can vanilla RNN suffer only from vanishing gradient or also from exploding gradient? — Both, but it is more prone to vanishing gradient.
10. What is the deeper issue: lack of expressive power or difficulty of optimization through time? — Primarily difficulty of optimization through time; memory representation is also structurally weak.

10 Summary: Key Points

- TF-IDF is useful for document-level signals but ignores sequence structure.
- Sequence models process token by token and maintain an evolving context.
- Vanilla RNN was one of the earliest deep learning solutions to this idea but suffers from vanishing and exploding gradients.
- The tanh derivative lies in $[0, 1]$; repeated multiplication through time causes vanishing.
- A Jacobian norm greater than 1 causes exploding gradients, but tanh saturation often offsets this, making vanishing more common.
- LSTM introduces a cell state C_t and a hidden state h_t, separating long-term memory from exposed working state.

- Three gates — forget, input, output — plus a candidate memory $\tilde{C}_t$ control information flow.
- The cell state update $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$ is additive, giving a direct gradient path and solving long-range learning.
- LSTM still processes sequentially, motivating architectures like the Transformer.